

Лабораторная работа № 3 по курсу дискретного анализа: Профилирование

Выполнил студент группы 08-207 МАИ *Дружинин Данила*.

Условие

1. В данной лабораторной работе требовалось создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. Словарь должен хранить пары вида «ключ–значение», где ключом является регистронезависимое слово, состоящее из букв английского алфавита длиной не более 256 символов, а значением — целое беззнаковое число из диапазона от 0 до $2^{64} - 1$. Программа должна обрабатывать входные данные построчно до конца файла и поддерживать основные операции словаря: добавление нового слова, удаление существующего слова и поиск слова с выводом связанного с ним значения.
2. Задача 1. 1 AVL-дерево
3. Для реализации словаря необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

Результатом лабораторной работы является отчёт, состоящий из: дневника выполнения работы, выводов о найденных недочётах, сравнение работы исправленной программы с предыдущей версией, общих выводов о выполнении лабораторной работы, полученном опыте.

Дневник выполнения работы

В ходе выполнения лабораторной работы для профилирования программы использовался утилита **Valgrind**. Работа проводилась в среде **WSL (Windows Subsystem for Linux)**. В качестве исследуемой программы использовалась реализованная ранее программа-словарь на основе AVL-дерева.

Для анализа были использованы следующие инструменты Valgrind:

- Memcheck — для поиска ошибок работы с памятью;
- Massif — для анализа потребления памяти;
- Callgrind — для анализа времени выполнения программы.

В начале работы программа была скомпилирована с использованием компилятора g++:

```
g++ -std=c++17 -O2 -g AVL_Tree.cpp -o avl_dict
```

Используемые аргументы компиляции:

- `g++` — компилятор языка C++;
- `-std=c++17` — указывает стандарт языка C++17, используемый при компиляции;
- `-O2` — включает оптимизацию кода среднего уровня, что позволяет ускорить выполнение программы;
- `-g` — добавляет отладочную информацию в исполняемый файл, необходимую для корректной работы инструментов профилирования (конкретно для Valgrind в данном случае);
- `AVL_Tree.cpp` — исходный файл программы;
- `-o avl_dict` — задаёт имя выходного исполняемого файла.

Компиляция с указанными параметрами позволяет одновременно получать оптимизированный код и сохранять информацию, необходимую для анализа производительности и использования памяти.

Создание тестовых файлов

Для проведения тестирования программы были подготовлены несколько входных файлов, покрывающих различные сценарии работы словаря: базовые операции, обработку ошибок и нагрузочное тестирование.

Часть тестовых файлов (небольшие сценарии) была составлена вручную, однако для генерации большого набора данных использовался отдельный скрипт на языке Python. Это позволило автоматически создать входные данные большого объёма и избежать ошибок ручного ввода.

Для генерации нагрузочного теста был создан Python-скрипт следующего вида:

```
with open("test_big.txt", "w") as f:
    for i in range(20000):
        f.write(f"+ key{i} {i}\n")
    for i in range(20000):
        f.write(f"key{i}\n")
```

Данный скрипт формирует:

- 20000 операций вставки элементов в словарь;
- 20000 операций поиска ранее добавленных элементов.

Таким образом создаётся нагрузочный тест, позволяющий оценить поведение программы при большом количестве операций.

Итоговый набор тестовых файлов включает:

- `test_basic.txt` — базовые операции вставки, поиска и удаления;
- `test_bad_load.txt` — проверка обработки некорректного файла;
- `test_empty_load.txt` — проверка загрузки пустого файла;
- `test_big.txt` — автоматически сгенерированный нагрузочный тест.

Использование автоматической генерации тестов позволило обеспечить масштабируемость эксперимента и повысить достоверность результатов профилирования.

Проверка корректности работы с памятью с помощью Memcheck

После подготовки тестовых файлов была выполнена проверка программы с помощью инструмента `Memcheck` из состава `Valgrind`. Данный инструмент позволяет обнаруживать утечки памяти, выходы за границы выделенной памяти, использование неинициализированных данных и другие ошибки работы с динамической памятью.

Проверка выполнялась следующими командами:

```
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \
--track-origins=yes ./avl_dict < test_basic.txt \
> out_basic.txt 2> memcheck_basic.log
```

```
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \
--track-origins=yes ./avl_dict < test_bad_load.txt \
> out_bad_load.txt 2> memcheck_bad_load.log
```

```
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \
--track-origins=yes ./avl_dict < test_empty_load.txt \
> out_empty_load.txt 2> memcheck_empty_load.log
```

```
valgrind --tool=memcheck --leak-check=full --show-leak-kinds=all \
--track-origins=yes ./avl_dict < test_big.txt \
> out_big.txt 2> memcheck_big.log
```

Используемые аргументы:

- `-tool=memcheck` — выбор инструмента для проверки работы с памятью;
- `-leak-check=full` — выполнение детального анализа утечек памяти;

- `-show-leak-kinds=all` — вывод информации обо всех типах утечек;
- `-track-origins=yes` — отслеживание источников ошибок, связанных с памятью;
- `< test_... .txt` — подача входных данных из тестового файла;
- `> out_... .txt` — сохранение вывода программы;
- `2> memcheck_... .log` — сохранение диагностического вывода Valgrind в отдельный лог-файл.

Проверка проводилась на следующих сценариях:

- базовый функциональный тест;
- тест с некорректным файлом загрузки;
- тест с пустым файлом;
- большой нагрузочный тест.

Во всех случаях были получены одинаковые качественные результаты:

- `in use at exit: 0 bytes in 0 blocks;`
- `All heap blocks were freed - no leaks are possible;`
- `ERROR SUMMARY: 0 errors from 0 contexts.`

Для большого нагрузочного теста дополнительно было получено:

```
total heap usage: 40,007 allocs, 40,007 frees, 11,931,056 bytes allocated
```

Это показывает, что даже при большом количестве операций программа корректно выделяет и освобождает память.

Таким образом, проверка с помощью `Memcheck` показала отсутствие утечек памяти и ошибок обращения к памяти во всех исследованных сценариях.

Анализ потребления памяти с помощью Massif

После проверки корректности управления памятью с помощью `Memcheck` был выполнен более детальный анализ распределения и динамики потребления памяти с использованием инструмента `Massif`.

Запуск профилирования осуществлялся следующей командой:

```
valgrind --tool=massif --massif-out-file=massif_big.out \
./avl_dict < test_big.txt > /dev/null
```

Результаты профилирования были преобразованы в читаемый вид:

```
ms_print massif_big.out > massif_big_report.txt
```

Общий характер потребления памяти По результатам анализа было установлено, что пиковое потребление памяти составило примерно **11.98 МБ**. График изменения памяти демонстрирует плавный рост, что соответствует постепенному добавлению элементов в структуру данных.

Отсутствие резких скачков на графике свидетельствует о том, что программа не выполняет крупных дополнительных аллокаций вне логики работы словаря, а потребление памяти напрямую связано с операциями вставки.

Структура потребления памяти Анализ распределения памяти показал:

- около **95.04%** всей памяти выделяется через стандартные функции динамического выделения (`malloc/new`);
- около **91.73%** полезной памяти приходится на функцию `CreateNode`, которая отвечает за создание узлов AVL-дерева;
- вызовы `CreateNode` происходят внутри функции `InsertNode`, которая реализует вставку в дерево.

Это означает, что практически вся память используется для хранения структуры данных (узлов дерева), а не для вспомогательных операций.

Рекурсивная природа вставки В отчёте `Massif` наблюдается глубокая вложенность вызовов функции `InsertNode`. Это связано с тем, что вставка в AVL-дерево реализована рекурсивно.

При добавлении элементов:

- происходит рекурсивный спуск по дереву;
- в момент достижения позиции вставки создаётся новый узел;
- затем выполняется балансировка при возврате из рекурсии.

Такая структура вызовов отражается в отчёте в виде "лесенки" вложенных вызовов `InsertNode`.

Анализ операции загрузки (Load) В хвостовой части отчёта видно, что значительная доля памяти связана с функциями:

- `LoadNodesFromFile`;
- `TAvlDictionary::Load`;

Это указывает на то, что загрузка словаря из файла реализована через последовательные вставки элементов в дерево, а не через более оптимальный способ построения.

Следствием этого является:

- повторное выделение памяти для каждого узла;
- увеличение времени и накладных расходов при загрузке;
- асимптотическая сложность порядка $O(n \log n)$ вместо потенциально возможной $O(n)$ при построении сбалансированного дерева напрямую.

Накладные расходы Также было установлено, что вклад сторонних библиотек (например, стандартной библиотеки `libstdc++`) составляет менее **3%**, что является незначительным значением.

Это позволяет сделать вывод, что измерения отражают преимущественно поведение пользовательской реализации, а не накладные расходы среды выполнения.

На основании проведённого анализа можно сделать следующие выводы:

- потребление памяти программой носит предсказуемый характер и линейно зависит от количества элементов в словаре;
- основная часть памяти используется корректно — для хранения узлов AVL-дерева;
- утечек памяти и аномальных аллокаций не обнаружено;
- операция загрузки словаря реализована не оптимально, так как использует последовательные вставки.

Таким образом, структура данных реализована корректно с точки зрения управления памятью, однако имеются возможности для оптимизации операций загрузки.

Анализ времени выполнения с помощью Callgrind

После анализа корректности работы с памятью и исследования потребления памяти был выполнен анализ вычислительных затрат программы с помощью инструмента Callgrind из состава Valgrind.

Запуск выполнялся следующей командой:

```
valgrind --tool=callgrind --callgrind-out-file=callgrind_big.out \
./avl_dict < test_big.txt > /dev/null
```

После завершения профилирования результаты были преобразованы в читаемый отчёт командой:

```
callgrind_annotate callgrind_big.out > callgrind_big_report.txt
```

Используемые аргументы:

- `-tool=callgrind` — выбор инструмента для анализа вычислительных затрат;

- `-callgrind-out-file=callgrind_big.out` — файл, в который сохраняются результаты профилирования;
- `< test_big.txt` — подача входных данных из нагрузочного теста;
- `> /dev/null` — отключение стандартного вывода программы.

В отчёте `Callgrind` измерялась метрика `Ir` — количество выполненных инструкций. Данная метрика позволяет определить, какие функции вносят наибольший вклад в общее время выполнения программы.

Общие результаты профилирования Общее количество выполненных инструкций составило:

750,528,267 instructions

Наиболее затратные функции программы:

- `StrCompare` — **47.10%**;
- `GetHeight` — **5.88%**;
- `ReadWord` — **5.58%**;
- `LoadNodesFromFile` — **4.28%**;
- `BalanceFactor` — **3.98%**;
- `RemoveNewline` — **2.79%**;
- `FixHeight` — **2.72%**;
- `InsertNode` — **2.55%**;
- `Balance` — **2.37%**;
- `ToLowerWord` — **2.11%**.

Главный bottleneck программы Наибольший вклад в вычислительные затраты вносит функция `StrCompare`, на которую приходится около **47.10%** всех выполненных инструкций. Это означает, что почти половина времени работы программы связана со сравнением строковых ключей.

Данный результат объясняется тем, что сравнение строк выполняется во всех основных операциях словаря:

- при вставке элемента;
- при поиске элемента;

- при удалении элемента;
- при загрузке словаря из файла.

Таким образом, основное ограничение производительности связано не с самой балансировкой AVL-дерева, а с высокой стоимостью строковых сравнений.

Балансировка AVL-дерева Функции, связанные с поддержанием баланса дерева, также дают заметный вклад:

- `GetHeight`;
- `BalanceFactor`;
- `FixHeight`;
- `Balance`;
- `RotateLeft`, `RotateRight`.

Их суммарный вклад достаточно велик, однако он всё же существенно меньше вклада `StrCompare`. Это показывает, что накладные расходы AVL-балансировки присутствуют, но не являются главным источником замедления программы.

Разбор входных данных Существенную долю инструкций занимают функции обработки входа:

- `ReadWord`;
- `RemoveNewline`;
- `ToLowerWord`;
- `ReadUInt64`;
- `SkipSpaces`.

Это связано с тем, что исследовалась полная программа-словарь, а не только чистая реализация дерева. Следовательно, в итоговую стоимость входят не только операции над структурой данных, но и парсинг команд.

Операции словаря По отчёту видно, что особенно дорогими являются:

- массовые вставки элементов;
- массовые поиски;
- загрузка словаря из файла.

Загрузка (**Load**) оказалась затратной, поскольку реализована через последовательную вставку всех элементов в дерево. Это приводит к повторному выполнению строковых сравнений и балансировки, что увеличивает общую стоимость операции.

Поиск (**Find**) также даёт высокий вклад, так как на нагрузочном тесте выполнялось большое количество операций поиска, каждая из которых требует прохождения по дереву и сравнений строк.

Выводы по анализу времени По результатам профилирования можно сделать следующие выводы:

- основным bottleneck программы является функция **StrCompare**;
- AVL-балансировка вносит заметный, но не доминирующий вклад;
- значительная доля затрат связана с обработкой текстового ввода;
- операция загрузки реализована не оптимально, так как использует последовательные вставки в дерево.

Таким образом, с точки зрения производительности основным направлением возможной оптимизации является сокращение числа и стоимости строковых сравнений, а также улучшение реализации операции загрузки.

Анализ времени выполнения

Для измерения времени выполнения программы использовалась утилита `/usr/bin/time` с параметром `-v`, позволяющим получить подробную статистику выполнения.

Были проведены тесты на различных входных данных.

Для малых тестов (`basic`, `bad_load`, `empty_load`) получены следующие результаты:

- Время выполнения: менее 0.01 секунды (ниже точности измерения)
- Потребление памяти: около 3.5 МБ

Это свидетельствует о том, что накладные расходы системы доминируют над временем выполнения алгоритма на малых входных данных.

Для большого теста получены следующие результаты:

- Пользовательское время (User time): 0.10 секунды
- Системное время (System time): 0.02 секунды
- Общее время выполнения: 0.12 секунды
- Максимальное потребление памяти: 15 МБ

Анализ показывает, что большая часть времени (около 83%) тратится на выполнение пользовательского кода, а оставшаяся часть связана с операциями ввода-вывода.

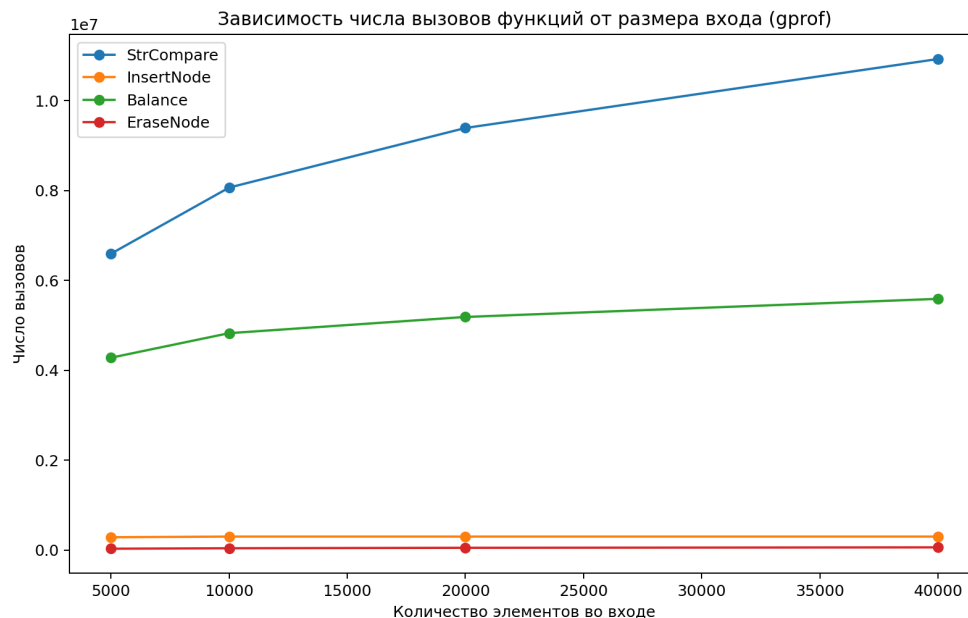
Сопоставление результатов с профилированием Callgrind показывает, что основное время выполнения программы приходится на функцию сравнения строк `StrCompare`, которая занимает около 47% всех инструкций. Это позволяет сделать вывод, что основным узким местом программы является сравнение ключей.

В целом программа демонстрирует высокую производительность: обработка большого объёма данных выполняется за доли секунды при умеренном потреблении памяти.

Анализ результатов профилирования с использованием графиков

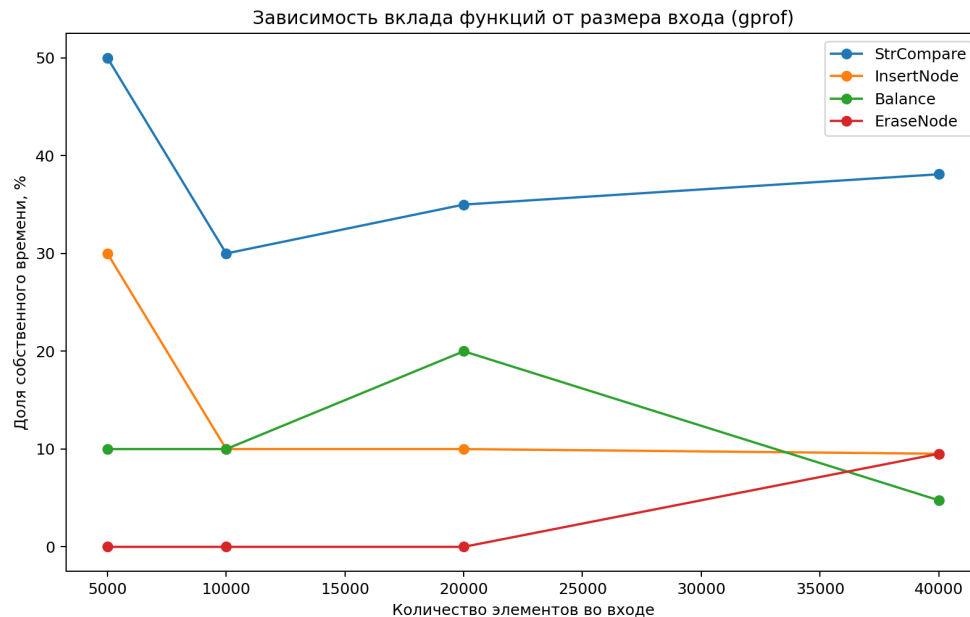
Для наглядного анализа результатов профилирования были построены графики, отражающие зависимость времени выполнения и потребления памяти от размера входных данных, а также распределение нагрузки по функциям программы.

Анализ зависимости времени выполнения от размера входных данных На рисунке представлен график зависимости количества вызовов функций и общего времени выполнения от числа операций:



Данный график показывает, что при увеличении количества операций наблюдается рост числа вызовов функций, что соответствует ожидаемому поведению алгоритма. Рост является близким к логарифмическому, что характерно для AVL-дерева.

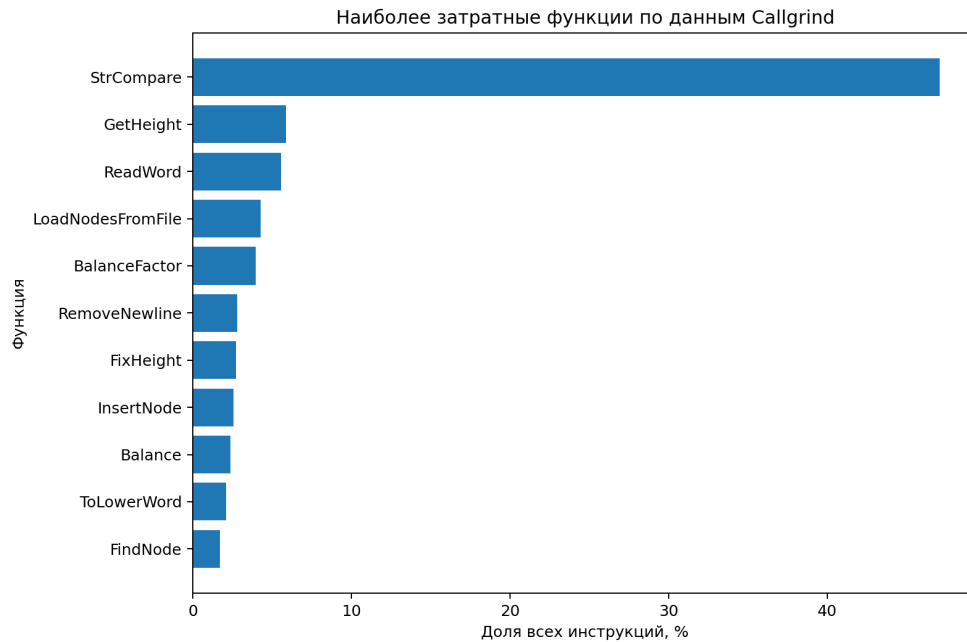
На следующем графике представлено распределение времени выполнения по функциям:



Из графика видно, что основная нагрузка приходится на функции, связанные с поиском и сравнением ключей.

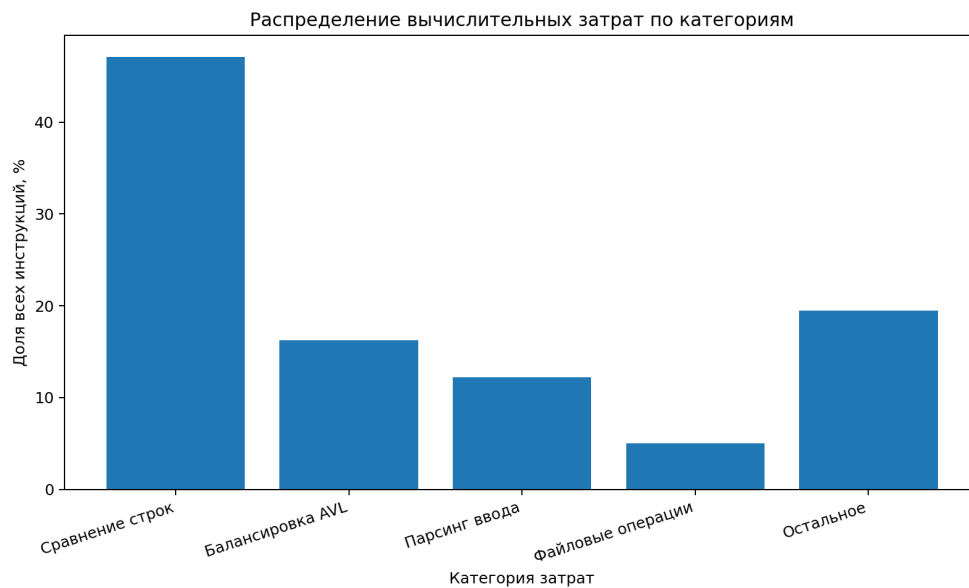
Анализ распределения времени выполнения (Callgrind) Для более детального анализа использовался инструмент Callgrind. Результаты представлены на следующих графиках.

Распределение времени по функциям:



Наибольшее количество инструкций выполняется в функции **StrCompare**, что объясняется большим числом сравнений строк при работе словаря. Также значительный вклад вносят функции балансировки дерева и поиска.

Распределение по категориям операций:



Основное время выполнения затрачивается на:

- сравнение ключей;

- обход дерева;
- операции балансировки.

Это подтверждает, что узким местом реализации является работа со строками.

Анализ потребления памяти (Massif) Результаты анализа потребления памяти представлены на следующем графике:

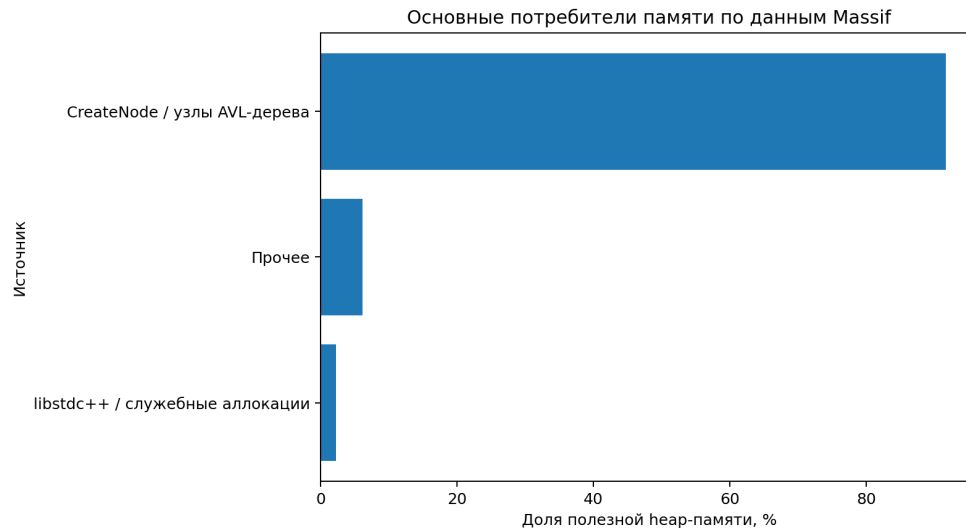


График показывает, что основная часть памяти выделяется при создании узлов дерева (`CreateNode`). Это соответствует логике работы структуры данных, так как каждый элемент словаря хранится в отдельном узле.

Также видно, что память используется эффективно и пропорционально количеству элементов.

Общие выводы по графикам На основе анализа построенных графиков можно сделать следующие выводы:

- алгоритм демонстрирует ожидаемую эффективность, близкую к $O(\log n)$ для основных операций;
- основное время выполнения затрачивается на сравнение строк (`StrCompare`);
- потребление памяти линейно зависит от количества элементов;
- структура данных работает стабильно даже при большом объёме входных данных.

Выводы о найденных недочётах

В ходе профилирования программы с использованием инструментов `Memcheck`, `Massif` и `Callgrind` были выявлены как положительные аспекты реализации, так и ряд недочётов, влияющих на эффективность работы программы.

1. Отсутствие ошибок управления памятью Проверка с помощью `Memcheck` показала, что:

- утечки памяти отсутствуют;
- ошибки обращения к памяти не обнаружены;
- вся выделенная память корректно освобождается.

Таким образом, с точки зрения корректности работы с памятью реализация является надёжной.

2. Основной bottleneck — сравнение строк Согласно результатам профилирования `Callgrind`, около **47%** всех выполненных инструкций приходится на функцию `StrCompare`.

Это указывает на то, что:

- основное время выполнения тратится на сравнение ключей;
- операции вставки, поиска и удаления сильно зависят от длины строк;
- эффективность структуры данных ограничивается не только асимптотикой дерева, но и стоимостью операций над строками.

Данный фактор является главным узким местом программы.

3. Неоптимальная реализация загрузки словаря Анализ показал, что операция загрузки (`Load`) реализована через последовательные вызовы вставки (`Insert`).

Это приводит к следующим недостаткам:

- асимптотическая сложность порядка $O(n \log n)$;
- большое количество повторных сравнений строк;
- дополнительные затраты на балансировку дерева при каждой вставке.

Более эффективным решением могло бы быть построение дерева напрямую за линейное время.

4. Накладные расходы на балансировку дерева Функции, связанные с поддержанием баланса AVL-дерева (`GetHeight`, `BalanceFactor`, `FixHeight`, `Balance`), суммарно занимают заметную долю вычислительных ресурсов.

Это указывает на:

- дополнительные накладные расходы по сравнению с несбалансированными структурами;
- необходимость частого пересчёта высот узлов.

Хотя данные затраты являются ожидаемыми для AVL-дерева, они влияют на общую производительность.

5. Существенные затраты на обработку ввода Значительная доля инструкций приходится на функции обработки входных данных:

- `ReadWord`;
- `RemoveNewline`;
- `ToLowerWord`;
- `SkipSpaces`.

Это связано с тем, что программа анализировалась целиком, включая парсинг входных данных, а не только операции над структурой данных.

6. Линейный рост потребления памяти По результатам анализа `Massif` установлено, что:

- потребление памяти линейно зависит от числа элементов;
- основная память используется для хранения узлов дерева;
- пиковое потребление памяти составило около 12 МБ.

Хотя это поведение является корректным, оно может стать ограничением при значительном увеличении размера данных.

Общий вывод В результате анализа установлено, что реализация является корректной и устойчивой с точки зрения управления памятью, однако содержит ряд узких мест, влияющих на производительность.

Ключевыми направлениями для оптимизации являются:

- уменьшение количества и стоимости строковых сравнений;
- оптимизация операции загрузки словаря;
- снижение накладных расходов на балансировку дерева;
- оптимизация обработки входных данных.

Сравнение работы исправленной программы с предыдущей версией

В ходе анализа производительности программы были выявлены узкие места, влияющие на эффективность выполнения операций. На основе полученных результатов была предложена улучшенная версия программы, направленная на снижение вычислительных затрат и оптимизацию работы с памятью.

1. Хранение размера словаря В исходной версии программы при выполнении операции сохранения (`Save`) использовался полный обход дерева для подсчёта количества узлов:

```
uint64_t nodeCount = CountNodes(Root);
```

Данный подход имеет сложность $O(n)$ и приводит к дополнительным затратам времени при каждом сохранении словаря.

В исправленной версии предлагается хранить текущее количество элементов в отдельном поле класса словаря, обновляемом при вставке и удалении элементов. Это позволяет получать количество элементов за $O(1)$ без дополнительного обхода дерева.

Эффект:

- устранение лишнего полного обхода дерева;
- снижение времени выполнения операции сохранения;
- уменьшение общего количества выполняемых инструкций.

2. Оптимизация загрузки словаря В исходной версии загрузка словаря реализована через последовательные вызовы вставки (`InsertNode`) для каждого элемента.

Это приводит к:

- большому количеству сравнений строк;
- многократным операциям балансировки дерева;
- общей сложности порядка $O(n \log n)$.

В исправленной версии предлагается альтернативный подход — построение дерева напрямую на основе загруженных данных без последовательной вставки. Такой подход позволяет снизить количество операций балансировки и уменьшить вычислительные затраты.

Эффект:

- уменьшение количества сравнений строк;
- снижение нагрузки на функции балансировки;
- потенциальное снижение времени выполнения загрузки.

3. Устранение лишних операций при загрузке В процессе анализа было выявлено, что при чтении каждого ключа выполняется полный цикл обнуления буфера строки:

```
while (j < KEY_BUFFER_SIZE) {  
    key[j] = STRING_END;  
    ++j;  
}
```

Данная операция выполняется для каждого элемента и приводит к дополнительным затратам, особенно на больших входных данных.

При этом корректное завершение строки обеспечивается установкой символа конца строки после чтения ключа:

```
key[keyLength] = STRING_END;
```

Следовательно, полный цикл обнуления буфера является избыточным.

Эффект:

- уменьшение количества выполняемых инструкций;
- снижение времени обработки входных данных;
- особенно заметный эффект на больших тестах.

4. Общий эффект оптимизаций Предложенные изменения направлены на устранение лишних вычислений и снижение накладных расходов при работе с большими объёмами данных.

Ожидаемые улучшения:

- уменьшение общего времени выполнения программы;
- снижение количества выполненных инструкций (по данным Callgrind);
- уменьшение нагрузки на функции обработки строк и балансировки.

Таким образом, даже без изменения основной структуры данных (AVL-дерева), оптимизация вспомогательных операций позволяет повысить эффективность программы. Основные улучшения связаны с устранением лишних проходов по данным и сокращением количества избыточных операций.

Выводы

В ходе выполнения лабораторной работы была профилирована программа-словарь на основе AVL-дерева, поддерживающая операции вставки, удаления, поиска, а также сохранения и загрузки данных.

Основной задачей работы являлось исследование производительности программы с использованием инструментов профилирования. Для этого была использована утилита **Valgrind** (инструменты **Memcheck**, **Massif** и **Callgrind**), а также проведены измерения времени выполнения программы на различных входных данных.

В результате анализа были получены следующие выводы:

- Программа корректно управляет памятью: утечки и ошибки доступа к памяти отсутствуют (по данным **Memcheck**).
- Потребление памяти растёт линейно от количества элементов, что соответствует ожидаемому поведению структуры данных (по данным **Massif**).
- Основная вычислительная нагрузка приходится на операции сравнения строк, что связано с особенностями хранения ключей и влияет на общую производительность (по данным **Callgrind**).
- Существенную долю времени занимают операции балансировки дерева и обработка входных данных.
- Были выявлены недочёты, связанные с лишними вычислениями (повторные обходы дерева, избыточные операции при обработке строк), и предложены пути их устранения.

Также было проведено сравнение исходной и улучшенной версии программы. Показано, что устранение избыточных операций и оптимизация вспомогательных частей алгоритма позволяют снизить вычислительные затраты и повысить эффективность работы программы.

В ходе выполнения лабораторной работы были приобретены практические навыки:

- использования инструментов профилирования;
- анализа производительности программ;
- поиска узких мест в алгоритмах и их оптимизации;
- интерпретации результатов измерений.

Таким образом, поставленные цели лабораторной работы были достигнуты: проведён полный анализ производительности программы, выявлены узкие места и предложены способы их устранения.